Lab 3: Handling Schema Evolution and Extracting Data from `_rescued_data` in Databricks Auto Loader

Lab Summary

In this lab, we test what happens when a **new file with a different schema** is uploaded to the Auto Loader source directory. The new file contains **additional columns** that were not present in the original schema.

When Auto Loader processes this file:

- The new columns are **not discarded**
- They are stored in a special column called `_rescued_data`
- `_rescued_data` stores the unexpected columns as a **stringified JSON**

We also learn how to **extract those new columns** by converting `_rescued_data` into a **structured JSON column** using `from_json()`.

Prerequisites

Before performing this lab, ensure the following:

- Databricks workspace is running
- Auto Loader pipeline from previous lab is active
- Bronze volume is created
- Streaming ingestion code is already configured
- Two batch files with **same schema** were already processed
- Destination data is stored in **Delta format**

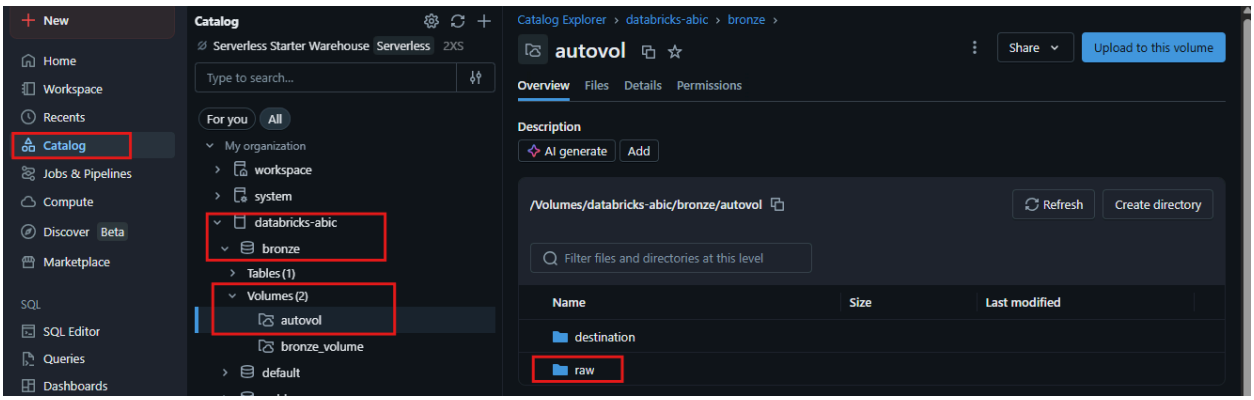
Steps to Complete Lab

Task 1: Upload a File with a Different Schema

Sub-task 1.1: Navigate to the Source Volume

Go to:

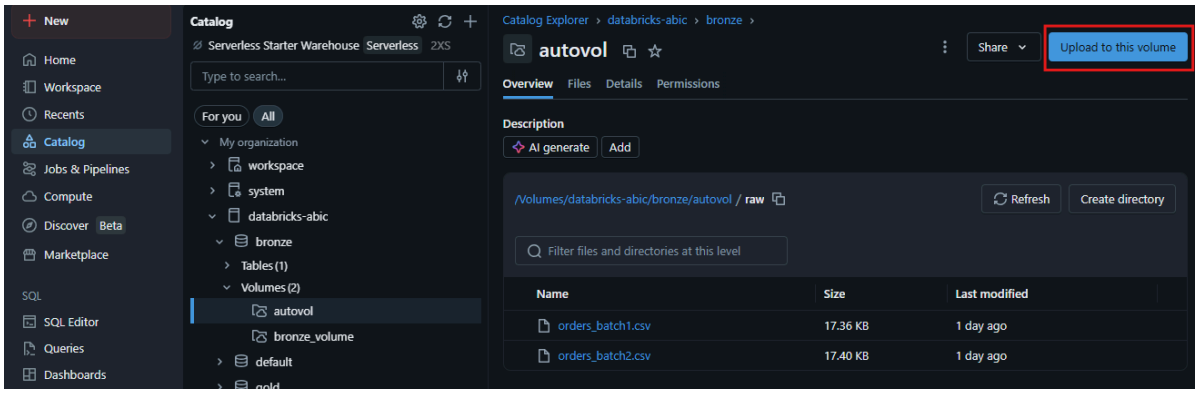
```
Catalog → Bronze → Volumes → autovol → raw
```



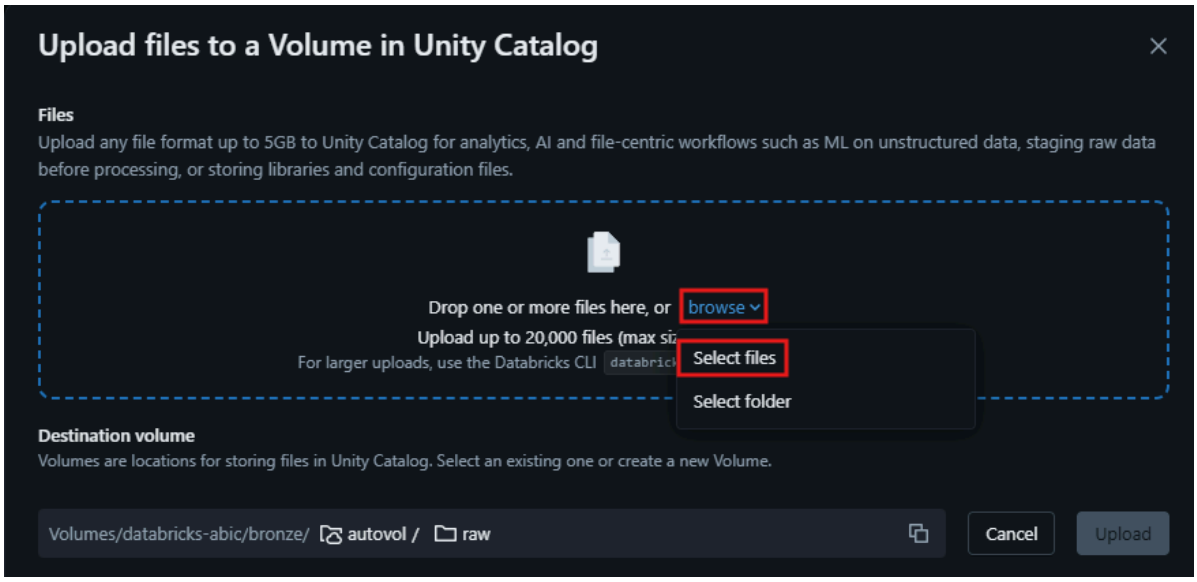
This is the **Auto Loader input directory**.

Sub-task 1.2: Upload the Third Batch File

1. Click **Upload to Volume**

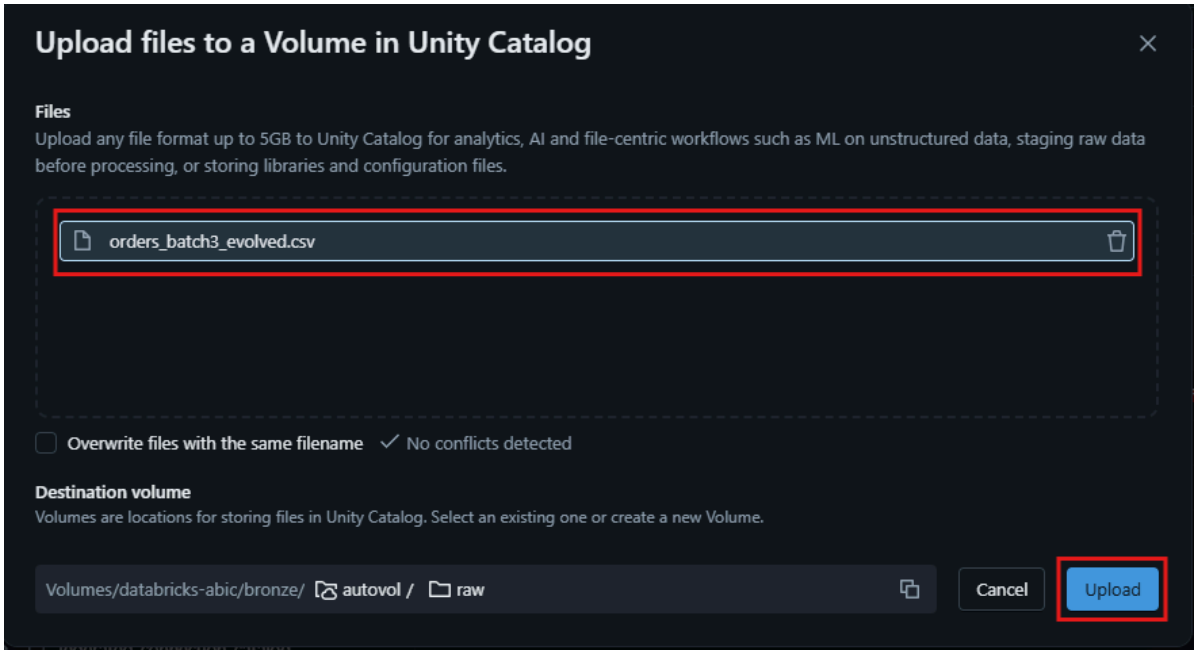


2. Select **Browse**



3. Choose the file:

```
orders_batch3_evolved.json
```



Important characteristic of this file:

- It contains **additional columns**
- The schema is **different from previous files**

Example new columns:

```
discount
payment_method
```

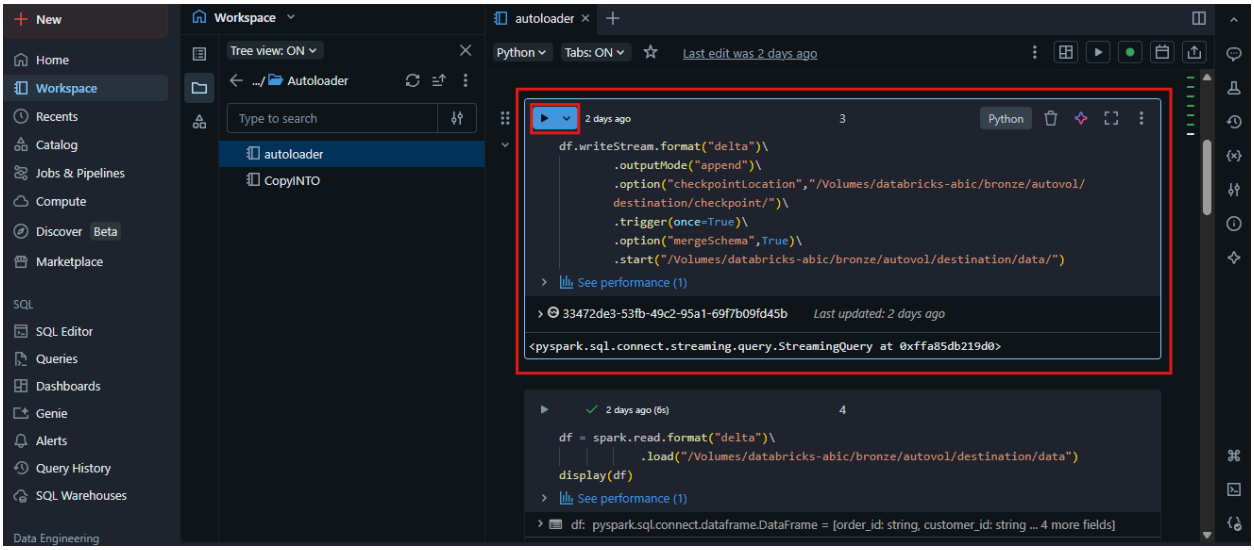
Upload the file to the **raw folder**.

Task 2: Run the Streaming Query Again

Sub-task 2.1: Execute the Streaming Write Code

Return to the notebook containing the Auto Loader pipeline.

Run the **same streaming write code cell again**.



Important:

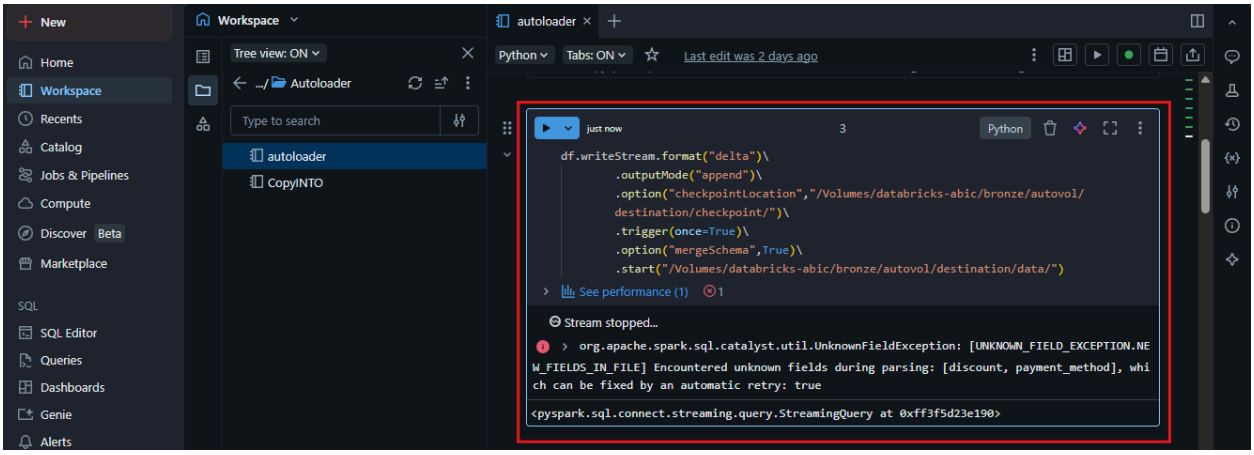
- Do **not recreate the pipeline**
- Just **rerun the existing streaming code**

Sub-task 2.2: Observe the Initial Error

When the query first runs, it may fail due to a **schema mismatch**.

Typical issue:

```
Schema mismatch detected
```

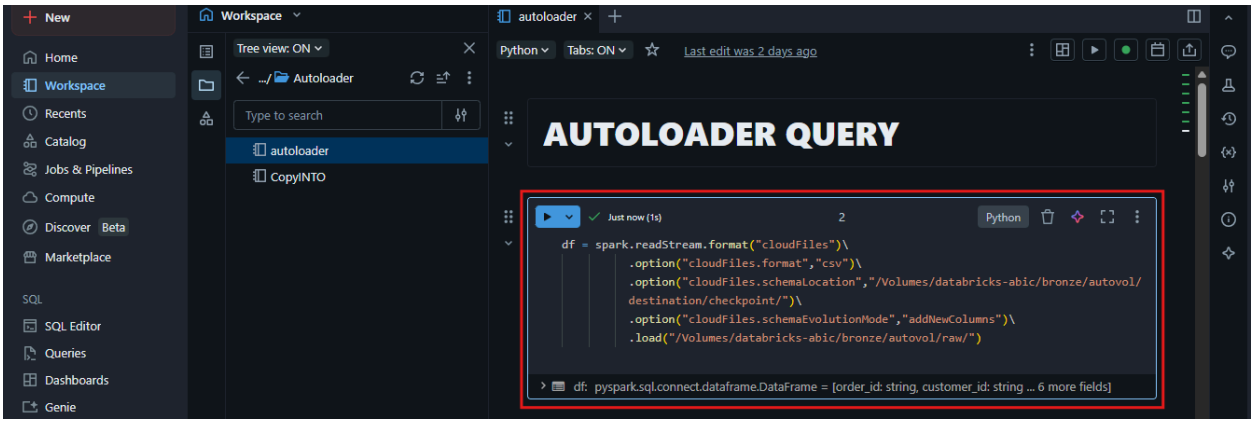


Why this happens:

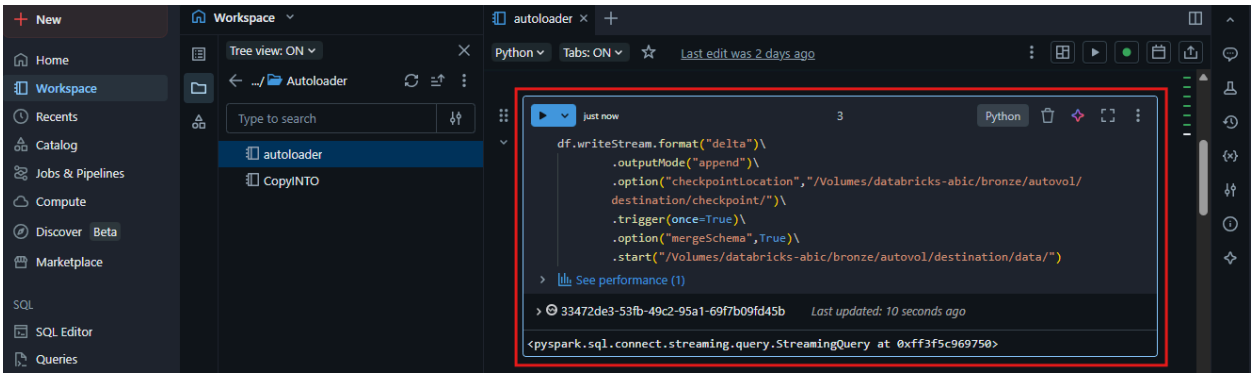
- The new file has **additional columns**
- The original schema does **not contain these fields**

Task 3: Query Reinitialization

When the query is run again:



1. Auto Loader reads the **latest schema from schema location**
2. It updates the stream schema
3. Processing continues successfully



Task 4: Verify the Processed Data

Sub-task 4.1: Read the Delta Destination

Run the following code to inspect the data.

```
df = spark.read.format("delta").load(destination_path)

display(df)
```

Sub-task 4.2: Observe the `_rescued_data` Column

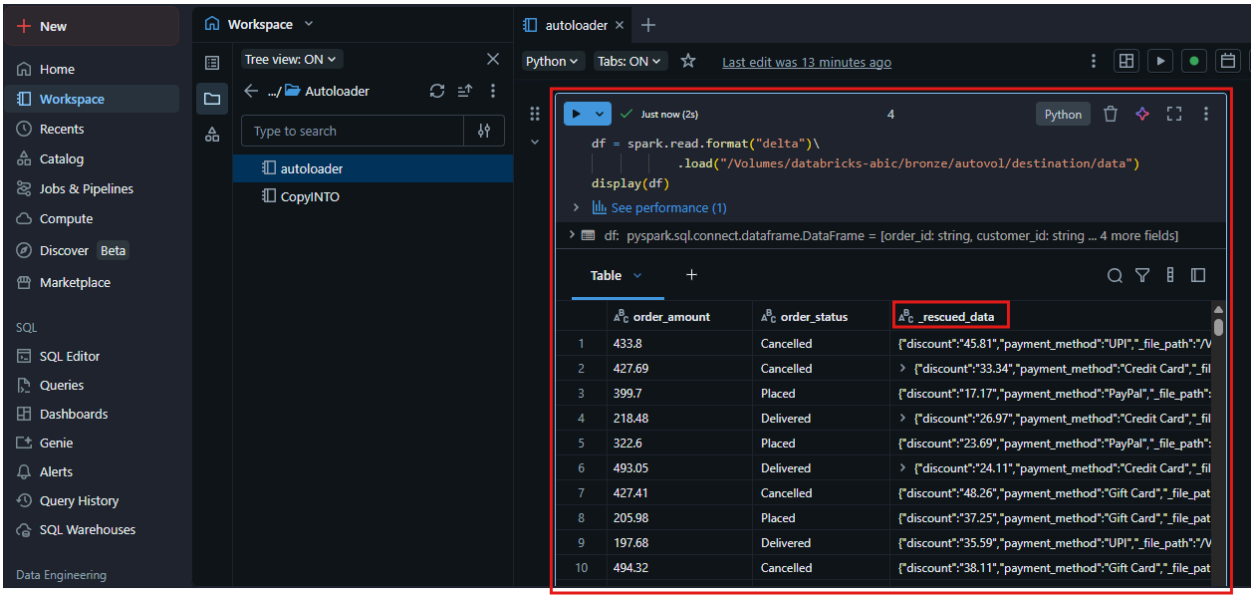
You will now see:

```
_rescued_data
```

Unlike previous runs, this column **contains values**.

Example:

```
{"discount":10,"payment_method":"card"}
```



Task 5: Understand the Structure of `_rescued_data`

Important point:

`_rescued_data` is **NOT actual JSON**

It is a **stringified JSON**.

Example data type:

```
StringType
```

This means we **cannot directly access fields** like:

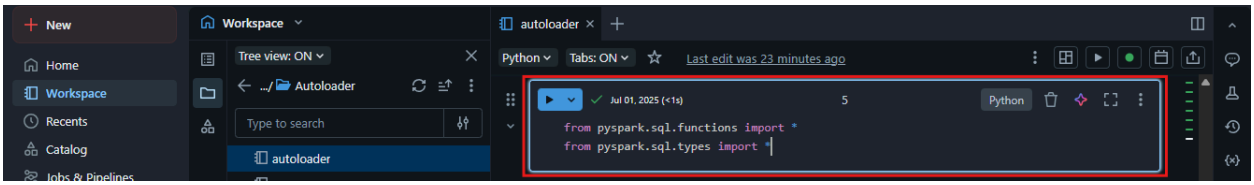
```
_rescued_data.discount
```

Instead, we must **convert it into a structured column first**.

Task 6: Convert `_rescued_data` to JSON

Sub-task 6.1: Import Required Functions

```
from pyspark.sql.functions import *
from pyspark.sql.types import *
```



Sub-task 6.2: Define the Schema for Rescued Data

We define the schema for the new columns.

```
rescued_schema = StructType() \
    .add("discount", StringType()) \
    .add("payment_method", StringType())
```

Sub-task 6.3: Convert `_rescued_data` into Struct

Use the `from_json()` function.

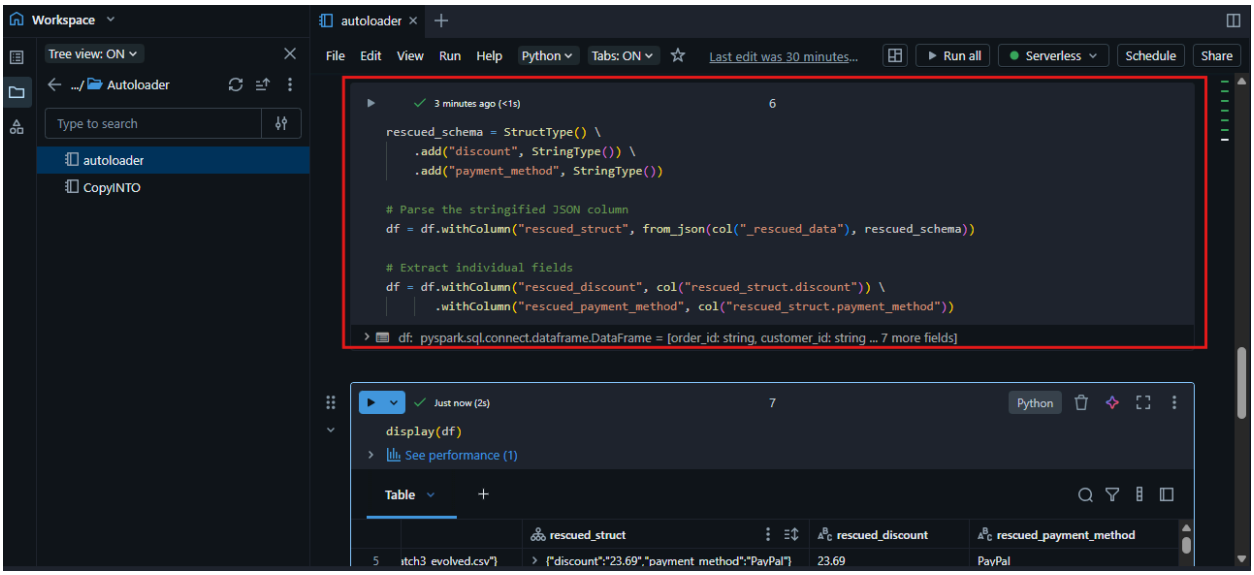
```
# Parse the stringified JSON column
df = df.withColumn("rescued_struct", from_json(col("_rescued_
data"), rescued_schema))
```

Now the column becomes **StructType instead of string**.

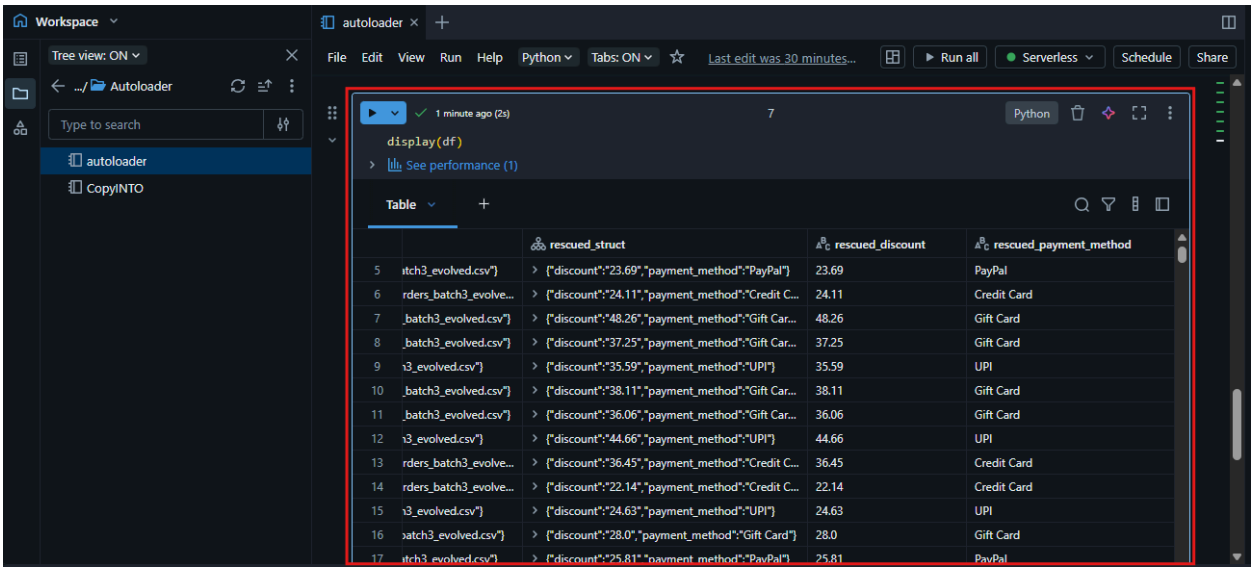
Task 7: Extract the New Columns

Once the column becomes **StructType**, we can extract fields.

```
# Extract individual fields
df = df.withColumn("rescued_discount", col("rescued_struct.di
scount")) \
       .withColumn("rescued_payment_method", col("rescued_str
uct.payment_method"))
```



Now the new columns appear as **normal DataFrame columns**.



Outcome

After completing this lab, you learned:

- How Auto Loader handles **schema evolution**
- How new columns are captured in `_rescued_data`
- Why `_rescued_data` is stored as **stringified JSON**

- How to convert it using `from_json()`
- How to extract additional columns into normal DataFrame columns

Key takeaway:

Auto Loader ensures **data is never lost during schema changes** by safely storing unexpected columns in `_rescued_data`.